Everything will be on GitHub in a single repo

In the past, we had milestone and other documents. GitHub has support for readmes, Wikis, and Release notes. I describe the structure that I want for those here.

Questions: email Dr. Rigby at peter.rigby@concordia.ca (include 490 in the subject)

- Anonymous feedback form
    - If something isn't working, please let me know, so I can change it

Important to pass 490: Each iteration, you must show your "substantial engineering design and implementation". This is the major requirement for 490, you must progressively show that you can design and code a real system. Failing early and trying things out are fine, so long as you document it on GitHub and show the code you tried and why it didn't work (you'll still get a good grade). In contrast, if you don't do anything substantial or can't show what you've done, you will fail the course even if the rest of your team passes.

General information and Intro Material

- Course outline
- Tentative Milestone dates
- The link to the intro lecture video, and slide link.
- Fill in your Project Overview and send it to `peter.rigby@concordia.ca` for approval ASAP and before Sept 15
    - If you don't have a project, check out these Potential projects
    - Medical projects: For 490, complying with this healthcare data act is usually too complex. The only way to deal with this appropriately is to get your stakeholder to handle this. If they cannot, then you'll need a different project.
- Deal with IP, License, and stakeholder agreement before Sept 30th
- You must use GitHub "projects" which are "an adaptable spreadsheet, task-board, and road map that integrates with your issues and pull requests on GitHub to help you plan and track your work effectively."
    - **Make sure that you share your project planning board with rigbypc**

Releases and Video Presentations

Please see this document

Table of Contents

GitHub Code, Issues/Stories, and Milestones

Please read the following closely. You don't want to lose marks because your repo isn't set up properly. You'll also want to read this: https://guides.github.com/features/issues/

I use examples from the Compass a 490 project from a prior year.

1. Create a GitHub account (students get free private repos)
    a. Ensure that your registered email address with GitHub is used for commits (otherwise your work will not be recorded)
    b. Create a GitHub repo for your project
        i. Name it using CamelCase
2. **New:** you must use GitHub "projects" which are "an adaptable spreadsheet, task-board, and road map that integrates with your issues and pull requests on GitHub to help you plan and track your work effectively." See here for instructions
3. GitHub milestones will be your **Iterations**
    a. Name them "Iteration 1, Iteration 2, Iteration 3 (Release 1)"
        i. We plan 2 Iterations in advance,
    b. Example
4. GitHub labels will be the names of your **features**
    a. Example
5. GitHub issues will be your **stories**
    a. Example of all the issues
    b. Example of a single issue
    c. **important:** When you open a story
        i. Give a short name for each story
        ii. Write the full story: As user X, I want to do Y, so I can accomplish Z
        iii. Write the number of story points
        iv. Write the priority or value
        v. Write the risk
        vi. Assign each story to a feature (label)
        vii. Assign the ones you are planning to work on to a Iteration (milestone)
        viii. Estimate the ideal time of the related tasks
        ix. Link all commits, arch diagrams, summaries of discussions, etc
        x. Write your unit tests
        xi. Get customer signoff (or add labels)
        xii. The prof and TA won't even look at a story until it has all of the above
6. When you make a commit

    a. Important: **When you make a commit make sure you use the right username and email** registered on github otherwise it won't show up in the project metrics
    b. Write a commit log that describes what you did
    c. Link it to a story (issue) by using #
        i. For example, Working on #87 ...
        ii. You can get sophisticated, closing issues from commits "fix #87"
    d. If you worked with other people you may want to use @username to mention that the other person contributed
    e. Example
7. When you finish an iteration or release
    a. use git tag Iteration1
    b. We need to be able to go back to this version
    c. **New:** You must describe your personal contribution to each iteration and release. If there is no information or only sporadic contributions, you will get zero for that iteration and release. See below for more details
8. Instead of a milestone document you will need a
   Project Wiki and Documentation

Readme

<CI information>

Release demos

This will be filled in with links to your demo videos for each release.

Project summary (max one paragraph)

Elevator pitch description at a highlevel

Developer getting started guide

What would a new developer need to do to get the system up and running?

Link to your project's board.

Wiki table of contents

Wiki

Each of the pages required for your project's wiki. You can have other pages that are appropriate to your project specifically.

Meeting Minutes

<Date>
Atendance: <people's names>
Absent: <people's names>
Note taker: <rotate person>
Notes: <bullet points>
Action items: <bullet points on what you will do, ideally linked to issues>

Risks

Describe in bullet points or max one paragraph the most significant risk to the project. Conclude this discussion with how you have attacked this risk first (link to code or stories to provide concrete evidence)

Write the risks that will kill your project!

User consent and end-user license agreement

Please include your consent and license agreement for users

Legal and Ethical issues

Describe in bullet points any legal or ethical issues. If they have been described above in the risks, simply note this. Including environment and discrimination against minority groups. Will this impact/disrupt existing jobs or careers.

Include links to your signed IP opt-out, license, stakeholder agreement, and NDA if applicable:

- Get your stakeholder to fill in the Stakeholder responsibilities, which also requires a liability limitation clause in the software license.
- You need to select a software license that assigns IP and copyright and removes all liability.
    - The simplest is the MIT license. Read it as it is 2 paragraphs. It allows anyone with the code to do anything (including sell), but ensures that you can't be held liable if your software doesn't work as expected.

    - Without a lawyer you can't make a real IP agreement, which is why I suggest MIT. Be cautious of company IP agreements.
    - More details on IP and copyright ownership
- Sign Concordia's IP opt-out
    - Essentially Concordia does not own any of the IP and is not liable for any of the work done for the course.
    - Note: if your stakeholder is a Concordia employee, you cannot sign this agreement, and the university will own your work. You can always use an MIT license that will allow you to do whatever you want with the software afterward.
- You are NOT required to have an NDA, but if your stakeholder justifies the need for one, this is the only one the university can use (NDA).
    - You can sign other NDAs, but be very careful when signing a company's NDA as it is often very restricting.

Economic

Describe how you are going to make money or have some other kind of measurable impact.

Budget

Create a table estimating your costs, eg cloud. If you get free compute, include this in a separate column.

If you spend any money, please keep your receipt as well as your visa statement showing that you paid for it. While there is no guarantee of being reimbursed, we will hopefully get around $200 per per project. Please keep receipts and credit card statements for purchases.

Personas

Please describe the personas that you will be developing the system for. Some advice on personas can be found here.

Diversity statement

Describe how your system will deal with equity and diversity.

Overall Architecture and Class Diagrams

Show us the layers in your system and your domain classes. You can also include individual class diagrams in your stories on GitHub
Put in your highlevel arch diagram (ie the major components)

For all existing arch diagrams, please only describe changes in your arch

Infrastructure and tools

For each library, framework, database, tool, etc

Name Conventions

List your naming conventions or just provide a link to the standard ones used online.

For example: Java naming conventions

Testing Plan and Continuous Integration

Discuss how you will test your system. Discuss continuous integration strategy.

Discuss how code/pull-requests will be reviewed and approved

You will use GitHub actions.

Security

What is your plan for making your system secure?

Performance

What is your plan for making your system performant?

Deployment Plan and Infrastructure

You should also have a deployment diagram and release strategy. You should also have a deployment plan and record.

- Release to stakeholder (at each signoff)
- Release to family and friends (get them to try it out)
- Alpha release (release to stakeholder's trusted clients)
- Beta release (wider but still limited to a set of early users)
- Full release (only exceptional teams will have a general release, but everyone should have a plan for the stakeholder to go forward with)

Missing knowledge and Independent Learning

What knowledge were you missing?

How did you acquire the knowledge?

Iteration and Release Notes

How to make a release on GitHub

Overall summary

Max four sentence paragraph describing main achievements.

Velocity and Contractor Estimate

Did you accomplish what you thought you would? What slipped? Was anything done early?

Estimate how much you would charge the stakeholder for this iteration

Retrospective

What went well?

What went wrong?

What improvements did/will you make?

Breakdown by individual

Use the automatically generated release notes as a starting point. Here is how to generate automatic release notes.

Then each contributor describes the contribution based on what has been done. If you do not have any commits and no description of your contribution, you will get zero for that iteration and release. 490 is iterative, you must show contributions for each iteration.

Note: this available to everyone on the project. Be truthful, it is part of the team lead's job to point out any inappropriate claims of work.